

INTERNAL BUILD REPORT

MTSAi Traffic Data Lake

What was built, how it works, and what's been verified in the Test environment

Reference document	TDR-004 / DL-PLAN-001, "MTSAi Traffic Data Lake" brief v0.1
Scope of this report	Phase 0, Phase 1 & Phase 2 (all six build steps), Test account only
Target AWS account	Test — 690293068614 (ap-south-1)
Repository	mtsai-datalake
Report date	21 September 2026

Phase 1 — Applied & Verified (Test)

Phase 2 — All 6 build steps done, running nightly

Phase 0 — Blocked on real DB access

Dev — Out of scope

What changed since the last report (15 September): the export job went from a locally-run binary to a real, unattended pipeline — built the container, deployed it, and watched it fire on its own schedule with no one triggering it. Added a second scheduled job that populates the curated zone automatically, and a backfill mode that processes a date range instead of one day at a time. Extended the synthetic source database with a year of additional history and ran a real 30-day backfill against it. Two more real bugs found and fixed along the way.

§ Contents

1	Summary
2	Target Architecture
3	Data Zones & Storage Layout
4	Consumer Isolation Model
5	Export & Curate Pipeline (Phase 2, all six steps)
6	Automation — Nightly Schedules
7	Backfill Mode & Extended History
8	CI/CD & Deployment Pipeline
9	What Has Actually Been Built
10	Verification Results
11	Bugs Found & Fixed (13 total)
12	Outstanding Items

1 Summary

MTSAI's traffic data (camera events, trip events, third-party feeds) lives in a single Postgres database (RDS) that also runs the live production application. Analytical queries over that data compete with production traffic, are expensive to keep in RDS long-term, and several teams (analytics, forecasting, dashboards, auditors) all need access without touching the primary database directly. This project builds a minimal, serverless data lake on AWS S3 to solve that — storing cold data in an open format, queryable via SQL, with per-team access isolation and a path to per-record deletion for compliance (DPDPA/GDPR/PIPEDA).

This report covers what has actually been **built and verified** in the Test AWS account, not just what's planned. Everything below reflects real infrastructure that exists today and has been tested with live AWS API calls — not a design document.

Since the previous version of this report (15 September), the export job stopped being something a person had to run by hand: it's now built into a container, deployed to ECS Fargate, and triggered nightly by EventBridge Scheduler — confirmed by directly observing a real unattended firing, not just a manual invocation. A second scheduled job now populates the curated zone automatically. A backfill mode processes a date range in one run instead of one date at a time, verified against a real multi-week range. Two more real bugs were found and fixed along the way, bringing the running total to 13.

2 Target Architecture

All components are AWS-managed and serverless except the export and curate jobs, which run as scheduled containers. Data flows one direction: from the operational database, into cold storage, out through a SQL query layer to each consumer.

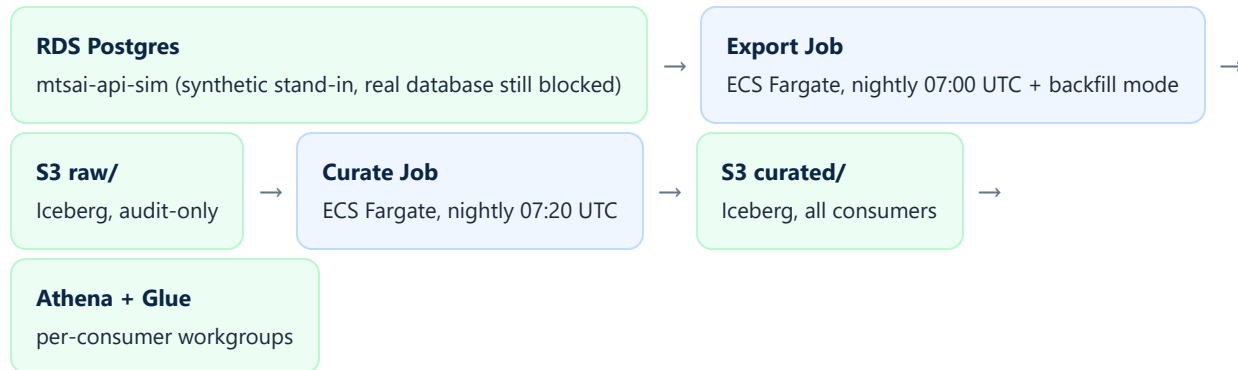


Figure 1 — end-to-end data flow. Green = live & verified, blue = scheduled/automated. Both jobs share one container image and IAM task role, differentiated by ECS container `entryPoint`.

Why two scheduled jobs, not one

The export job's only responsibility is getting Postgres data into `raw/` faithfully — it never touches `curated/`. The curate job runs 20 minutes later (ample margin; a normal run finishes in under a minute) and does a straight Athena `INSERT INTO ... SELECT ... FROM` from raw into curated — no staging table needed, since both sides are already Iceberg tables. Keeping these as two small, single-purpose jobs instead of one combined job matches the brief's own two-zone model and keeps each job's IAM footprint scoped to what it actually does.

3 Data Zones & Storage Layout

The lake uses two zones, per bucket `mtsai-datalake-test-690293068614-ap-south-1` :

Zone	Purpose	Who can read it	Format
<code>raw/</code>	Faithful, unmodified copy of source rows. Audit/replay layer.	audit workgroup only	Apache Iceberg
<code>curated/</code>	Schema-stable copy of raw, ready for analytics (v1: passthrough, not yet cleaned/deduped).	All consumers	Apache Iceberg
<code>athena-results/<consumer>/</code>	Per-consumer query output, 7-day expiry.	Only that consumer	CSV/metadata
<code>export-manifests/</code>	Reconciliation & audit trail for each export/curate/backfill run.	Export/curate jobs only	JSON

Apache Iceberg (not plain Parquet folders) supports row-level deletes — required for per-person erasure requests — plus schema evolution and time travel, without locking the lake into one query engine.

4 Consumer Isolation Model

Every consumer gets its own Athena workgroup and its own IAM role — never a shared identity. Isolation was verified by actually assuming each role via STS and running real Athena queries, not by switching workgroups while signed in as an admin (which tests nothing — the workgroup selection doesn't change which identity authorizes the query).

Role	Query against curated	Query against raw	Expected raw access?	Result
<code>analytics</code>	SUCCEEDED	DENIED	No	Correct
<code>forecasting</code>	SUCCEEDED	DENIED	No	Correct
<code>audit</code>	SUCCEEDED	SUCCEEDED	Yes	Correct

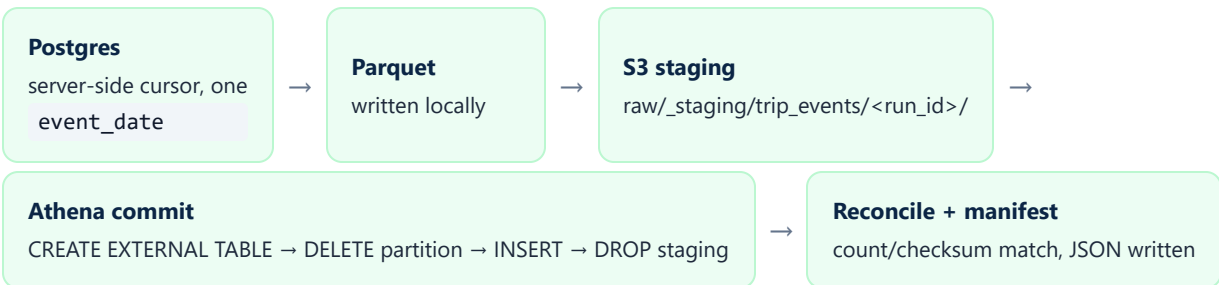
Repeatable via `runbooks/isolation-check.ps1` , committed to the repo.

5 Export & Curate Pipeline (Phase 2, all six build steps)

The brief's Phase 2 lists six build steps. All six are now built:

#	Step	Status
1	Export job in Go: read from Postgres, write Parquet, commit via Athena	Built & verified
2	Idempotency — re-running a date replaces, not duplicates	Built & verified
3	Reconciliation — row count + checksum compare, fails on mismatch	Built & verified
4	ARM64 container on ECS Fargate, scheduled nightly	Live & observed firing
5	Backfill — run over a date range, record throughput	Built & verified
6	Curated layer — scheduled query into curated tables	Live & verified

How a single-date export run works



Backfill mode (step 5)

`EXPORT_START_DATE / EXPORT_END_DATE` (instead of the single-date `EXPORT_DATE`) run the exact same per-date pipeline above once per day in the range — a "chunk" is one day, matching the Iceberg partition granularity. A single date's failure doesn't abort the rest of the range (so throughput can still be measured), but the overall run still fails if anything failed. Each date gets its normal manifest, plus a new aggregate summary at `export-manifests/backfill/<start>_<end>/summary.json` .

Curated layer (step 6)

A second binary, `cmd/curate` , in the same container image. Unlike the export job, it never touches Parquet or S3 directly — both sides are already Iceberg tables, so it's a straight cross-database `DELETE + INSERT INTO ... SELECT ... FROM` via Athena. v1 curated is a schema-stable passthrough copy of raw (real cleaning/dedup logic is future work), created once by hand via CTAS (`sql/curated/trip_events_curated.sql` , committed for reproducibility this time).

6 Automation — Nightly Schedules

Both jobs run as ECS Fargate tasks, triggered by EventBridge Scheduler — no cron server, no always-on infrastructure. Same container image, same task role; only the ECS container `entryPoint` differs.

Schedule	Expression	Local time	Target
mtsai-datalake-test-nightly-export	<code>cron(0 7 * * ? *)</code>	12:30 PM IST	mtsai-datalake-test-export task def
mtsai-datalake-test-nightly-curate	<code>cron(20 7 * * ? *)</code>	12:50 PM IST	mtsai-datalake-test-curate task def

A real unattended firing was directly observed, not just simulated

What this proves

- ECS task appeared under the cluster's Tasks tab with `startedBy: chronos-schedule/mtsai-datalake-test` — EventBridge's own signature, distinct from any manual invocation
- Ran clean end-to-end: exit code 0
- Zero rows is expected, not a bug — no `EXPORT_DATE` override defaults to "yesterday," and the seeded fixture data doesn't move with the clock

What hasn't been observed yet

- Seven consecutive nights with zero reconciliation failures (brief's own "done when" bar) — only one real night so far
- The curate schedule's own unattended firing (verified so far only via manual invocation of the deployed task)

7 Backfill Mode & Extended History

Every export run up to this point covered only a handful of dates the original seed happened to produce. To exercise the backfill mechanism at something closer to real scale, `tools/seed-api-db` was extended with an additive-only mode (`SEED_EXTEND_HISTORY=1`) that inserts `trip_events` rows across an older, disjoint date range — without truncating anything, so every already-verified date/checksum stays intact.

Metric	Value
New distinct dates added to mtsai-api-sim	271 (2025-09-21 to 2026-06-18)
Backfill range run into the lake	2026-05-20 to 2026-06-18 (30 days)
Result	30 succeeded, 0 failed, 3,198 total rows, 260s wall clock
Independent verification	Athena <code>COUNT(*)</code> over the same range → 3,198, matching the backfill summary manifest exactly

The remaining ~241 newly-seeded dates are seeded but not yet exported — running the rest is mechanically identical, just a longer-running invocation of the same command (documented in `runbooks/backfill.md`).

Honest scope note: this is not "full production history" — real `mtsai-api` access is still blocked (Phase 0), so this backfill exercised the mechanism against the synthetic stand-in only. What's proven is that the mechanism itself works correctly at multi-week scale, not that it's been run against real data.

8 CI/CD & Deployment Pipeline

Infrastructure is provisioned via Terraform, applied through GitHub Actions using short-lived OIDC credentials — no long-lived AWS keys stored anywhere in CI. Unchanged since the last report.

Workflow	Trigger	Approval gate	Purpose
<code>bootstrap.yaml</code>	Manual	None	Creates the Terraform state S3 bucket + DynamoDB lock table, once per account
<code>deploy.yaml</code>	Manual	None	Runs <code>terraform apply</code> or <code>destroy</code> immediately
<code>deploy-locked.yaml</code>	Manual	GitHub Environment reviewer	Same as above, but pauses for approval — used for anything destructive
<code>debug-oidc.yml</code>	Manual	None	Decodes the OIDC token's claims to debug an IAM trust policy, without calling AWS

9 What Has Actually Been Built

Phase	Status	Notes
Phase 0 — Discovery	Blocked	No real <code>mtsai-api</code> DB access from this environment yet; deliverables filled in against a synthetic stand-in only.
Phase 1 — Foundation	Applied & verified (Test)	S3 bucket, KMS key, Glue catalog, 3 Athena workgroups + IAM roles, ECS scaffolding, alarms — all live in Test.
Phase 2 — Export pipeline	All 6 build steps done	Running nightly, unattended, in AWS. Brief's own time/volume-based "done when" (7 nights, full backfill, 20 curated queries) not yet met.
Phase 3 — Governance & erasure	Not started	—
Phase 4 — Postgres trim	Not started	Depends on Phase 2/3.

Live/working in Test (690293068614)

- S3 lake bucket (+ SSE-S3 logs bucket) · KMS key
- Glue `test_raw` / `test_curated` databases, both with real Iceberg tables
- 3 Athena workgroups with scoped IAM roles
- ECS cluster + **two** task definitions (export, curate), real ECR image
- Two EventBridge schedules, one observed firing for real
- SNS/CloudWatch failure alerting
- Backfill mode, verified against a real 30-day range
- `mtsai-api-sim` RDS instance with a year of seeded history

Deliberately not built yet

- Dev environment (out of scope per direction)
- Real `mtsai-api` access (Phase 0 blocker)
- Handling a second table
- Real cleaning/dedup logic in the curated layer (v1 is a passthrough)
- AWS Budgets / cost alerts (needs an alarm email)
- Plan-on-PR / apply-on-merge CI (still manual-dispatch only)
- KMS key policy restriction (relies on IAM alone)

10 Verification Results

Every claim in this report was checked by actually running the system and independently re-confirming the result — via Athena, CloudWatch Logs, and S3 manifests — not by trusting a program's own exit code alone.

Check	Result
Manual invocation, real deployed export task, 2026-07-07	76 rows, checksum 177962 — matches every prior run of the same date
Manual invocation, real deployed curate task, 2026-07-08	55 rows, checksum 141034 — matches raw exactly, independently reconfirmed via Athena
Idempotency — curate re-run, same date	Still 76 rows, not 152
Real unattended nightly firing, 2026-09-21 07:00 UTC	<code>startedBy: chronos-schedule/...</code> , exit code 0
30-day backfill, real deployed export task	30/30 succeeded, 3,198 rows, matches Athena's independent count exactly
Extended seed data — original dates unaffected	2026-07-07's count/checksum re-checked unchanged (76/177962) after the additive seed ran

11 Bugs Found & Fixed

Thirteen real bugs found across this project so far, none caught by `terraform plan`, `go build`, or code review — all found only by actually exercising the deployed system with real API calls.

Platform / isolation & storage (7)

- 1. S3 key enumeration leak.** `s3:ListBucket` granted on the bare bucket ARN let every consumer list keys under `raw/` regardless of read permission. Fixed with an `s3:prefix` -scoped statement per zone.
- 2. Missing `s3:GetBucketLocation` (consumer roles).** Athena calls this on the results bucket before any query — missing it produced "Unable to verify/create output bucket" for every consumer.
- 3. Tag-based Glue permissions don't work for hand-created tables.** Tables made via Athena DDL/CTAS never receive Terraform's resource tags, silently denying `glue:GetTable`. Fixed with explicit ARN scoping instead of tags.
- 4. Glue's hierarchical authorization.** Reading a table needs an explicit Allow at catalog, database, *and* table level — the catalog-level grant was missing for read actions.
- 5. S3 access logging silently did nothing.** S3 server access logging doesn't support an SSE-KMS destination; the lake bucket logged to itself and delivered nothing. Fixed with a separate SSE-S3 logs bucket.
- 6. ...and that logs bucket had no delivery permission either.** Object Ownership "Bucket owner enforced" blocks the classic ACL grant; no bucket policy existed as the alternative. Fixed with an explicit policy scoped via `aws:SourceArn` / `aws:SourceAccount`.
- 7. Missing `s3:GetBucketLocation` (export task's own role).** The same gotcha as #2, but on the export task's role, found only when the deployed container actually tried to commit to Iceberg for the first time — first automated run failed at the very last step despite the read/upload steps succeeding.

Export & curate pipeline (6)

- 8. Athena's `WITH (...)` table syntax is CTAS-only.** Registering existing S3 files needs classic Hive DDL (`CREATE EXTERNAL TABLE ... STORED AS PARQUET LOCATION ...`), not the Trino `WITH` form.
- 9. `IF NOT EXISTS` breaks with `EXTERNAL`.** Undocumented, confirmed empirically. Dropped it — each staging table name already has a unique run-ID suffix.
- 10. `parquet-go` mis-encodes dates.** The library's `time.Time` writer only correctly handles the `Timestamp` logical type; `Date` silently wrote garbage. Root-caused via the library's own source, fixed with a hand-encoded days-since-epoch `int32`.
- 11. Export-task role had nowhere to write Athena results.** No S3 permission for `athena-results/export/*` — every query would have failed.
- 12. Export-task role couldn't clean up after itself.** Missing `glue:DeleteTable` for dropping the throwaway staging table each run creates.
- 13. `sslmode=disable` against RDS.** Worked fine against local Docker Postgres (no SSL configured at all) but RDS rejects plaintext connections outright. Fixed by switching to `sslmode=prefer`, which negotiates SSL when available and falls back when not — the same code now works unmodified against both.

All thirteen fixes are applied and re-verified in Test as of this report.

12 Outstanding Items

Item	Why it's not done
Seven consecutive nights, zero reconciliation failures	Time-based bar from the brief's own Phase 2 "done when" — only one real night observed so far, this just needs to accumulate.
Full history backfilled	Mechanism proven at 30-day scale; ~241 of 271 newly-seeded dates remain, and real <code>mtsai-api</code> history doesn't exist yet either way.
20 baseline queries rewritten against curated, with execution time/bytes scanned	Blocked on Phase 0 — those 20 queries don't exist yet since Phase 0 is still against the synthetic stand-in.
Real cleaning/dedup logic in the curated layer	v1 curated is honestly a schema-stable passthrough copy of raw; the brief only asked for "simple CTAS" for this slice.
Postgres read access (real <code>mtsai-api</code>)	No connection details or DB client available yet — the root blocker behind Phase 0 and several of the items above.
Second table / schema-generic export	Still hardcodes <code>trip_events</code> ; config file is real but not yet driving generic Parquet schema generation.
AWS Budgets / cost alerts	Needs an alarm email and confirmation that cost-allocation tags are active org-wide.
Plan-on-PR / apply-on-merge CI	Current workflows are manual-dispatch only.
KMS key policy restriction (Section 6)	Currently relies on IAM alone rather than a resource-level key policy limited to specific roles.

Generated from the live state of the `mtsai-datalake` repository and the deployed Test AWS account (690293068614) as of 21 September 2026. Diagrams reflect verified behavior, not aspirational design. Companion to `MTSAi-Data-Lake-Flow.pdf` (plain-language flow) and `MTSAi-Data-Lake-Phase2-Session-Log.pdf` (the 15 September session's literal command log — a dated historical record, not updated here).